OBSERVABILIDAD EN SPRING BOOT

Prometheus · Loki · Grafana

Juan Carlos Tobón Montoya 160004937

¿QUÉ ES OBSERVABILIDAD?

Es la capacidad de entender el estado interno de un sistema a partir de las señales que emite, incluso ante preguntas que no anticipaste.

Viene de la teoría de control (Kalman, 1960): un sistema es observable si sus salidas permiten inferir su estado.

En software esas salidas son métricas, logs y trazas. La prueba de fuego: si para entender un problema nuevo tienes que agregar un log y redesplegar, te falta observabilidad.

MONITOREO VS. OBSERVABILIDAD

Monitoreo responde «¿está funcionando?». Vigila problemas que ya conoces con umbrales y alertas. Ejemplo: la CPU pasa del 80% y suena una alerta.

Observabilidad responde «¿por qué se comporta así?». Te deja investigar problemas nuevos con telemetría rica. Ejemplo: el p99 de /pagos sube solo cuando responde una pasarela.

¿CUÁNDO USARLA?

SÍ, DESDE EL DÍA UNO

- Microservicios o sistemas distribuidos
- Dependencias externas (pagos, APIs)
- SLOs o contratos que cumplir
- Varios equipos sobre un mismo sistema
- Despliegues frecuentes

CON MESURA

- Prototipo o monolito pequeño: Actuator y logs JSON alcanzan
- Jobs batch: mide éxito y duración
- La telemetría cuesta: disco y cardinalidad
- Empieza por RED en los endpoints críticos

Regla práctica: cuanto más distribuido es el sistema y más dependencias externas tiene, más falta hace.

LOS TRES PILARES

MÉTRICAS

¿Cuánto?

Números agregados en el tiempo: cuánto, qué tan rápido, qué porcentaje. Baratas e ideales para alertar.

LOGS

¿Qué pasó?

Eventos con contexto: qué pasó en este request. Si son JSON, se consultan como datos.

TRAZAS

¿Dónde?

El recorrido de un request entre servicios: dónde se fue el tiempo. Clave en microservicios.

La métrica lleva al log. El log lleva a la traza.

¿POR QUÉ IMPORTA?

- Los sistemas de hoy fallan de formas nuevas: microservicios, colas y APIs de terceros. No puedes anticipar cada fallo con un check.

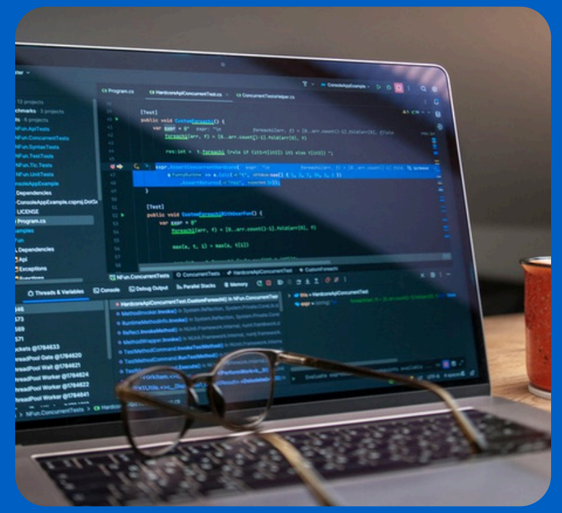
- El objetivo es bajar el MTTD y el MTTR: detectar antes y recuperar más rápido, decidiendo con datos y no por intuición.

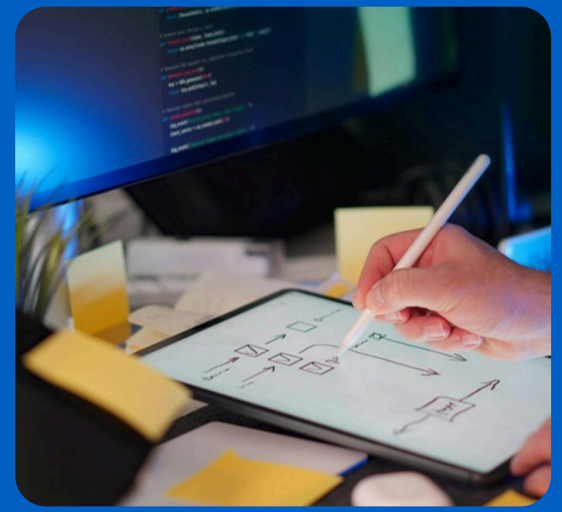
SLI, SLO Y PRESUPUESTO DE ERROR

- SLI, lo que mides. Ejemplo: el porcentaje de pagos que responden en menos de 500 ms.
- SLO, la meta. Ejemplo: el 99% de los pagos por debajo de 500 ms en una ventana de 30 días.
- Presupuesto de error, el 1% restante. Si se agota, la estabilidad va antes que las nuevas funcionalidades.

CAZAR CUELLOS DE BOTELLA

- Detectar: RED muestra el síntoma. Qué endpoint se volvió lento o falla, y desde cuándo.
- Acotar: USE y las trazas dicen dónde. ¿CPU saturada, pool agotado o una dependencia externa?
- Explicar: los logs dan la causa con contexto. Corriges y confirmas con las mismas métricas.



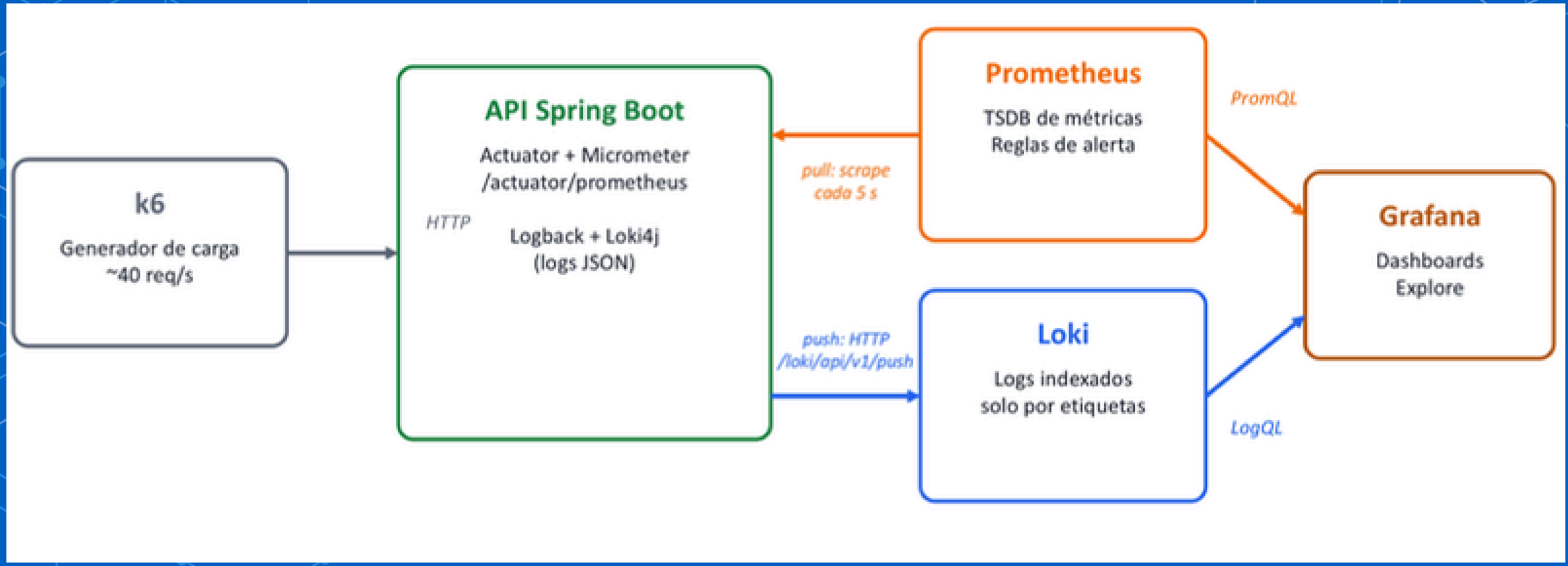


LA DEMO: UNA TIENDA CON TRAMPAS

- Una mini tienda en Spring Boot con tres problemas escondidos a propósito.
- Pagos depende de una pasarela externa, y pedidos tiene un endpoint con N+1 y su versión corregida.
- La usarán en el taller: escanea el QR para abrir el repositorio.

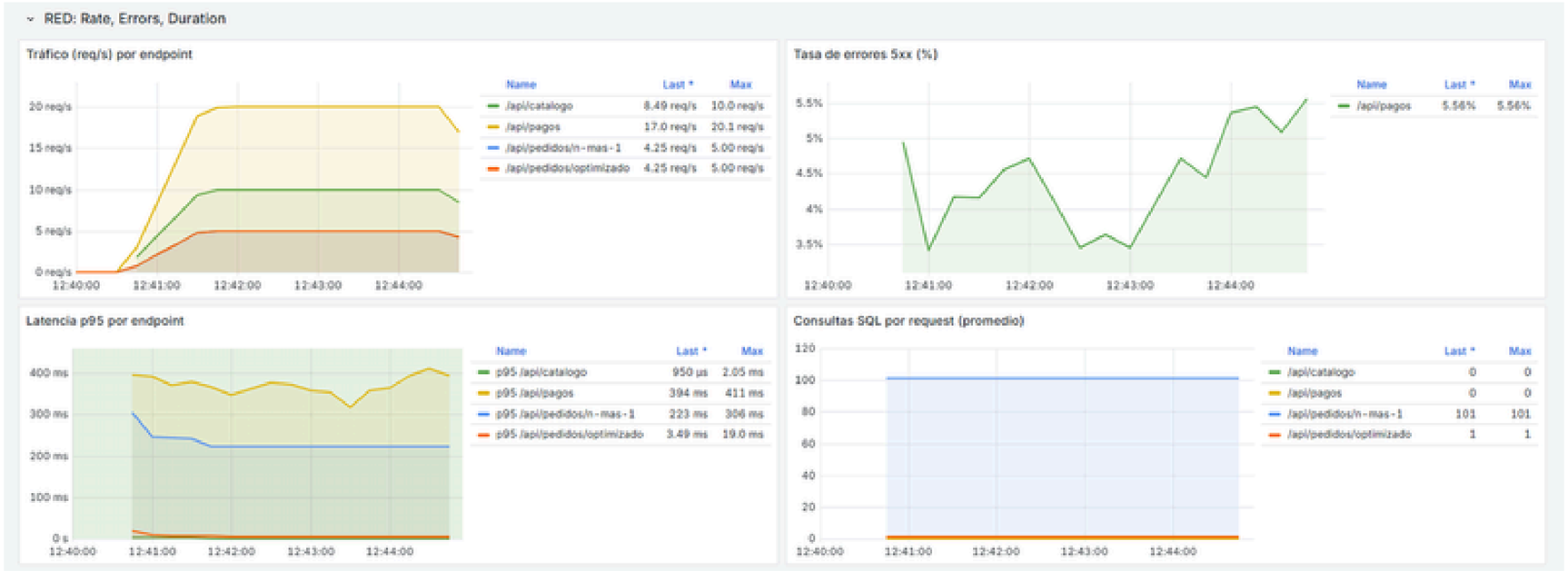
Repositorio de la demo

ARQUITECTURA DE LA DEMO



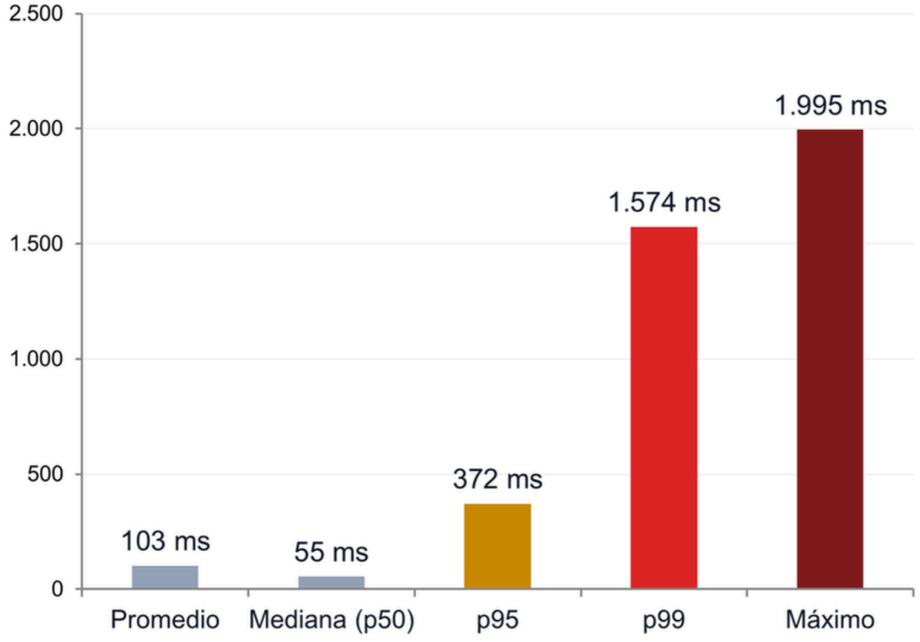
Prometheus va por las métricas cada 5 s (pull). Los logs JSON viajan a Loki (push). Grafana consulta ambos con PromQL y LogQL. Todo se levanta con docker compose.

EL DASHBOARD EN GRAFANA



Datos reales de la prueba de carga: solo /api/pagos tiene errores, su p95 ronda los 400 ms y /api/pedidos/n-mas-1 hace 101 consultas SQL por request.

CUELLO 1: EL PROMEDIO MIENTE



- En /api/pagos el promedio es 103 ms, pero el p99 llega a 1.574 ms: 1 de cada 100 pagos tarda más de 1,5 segundos.

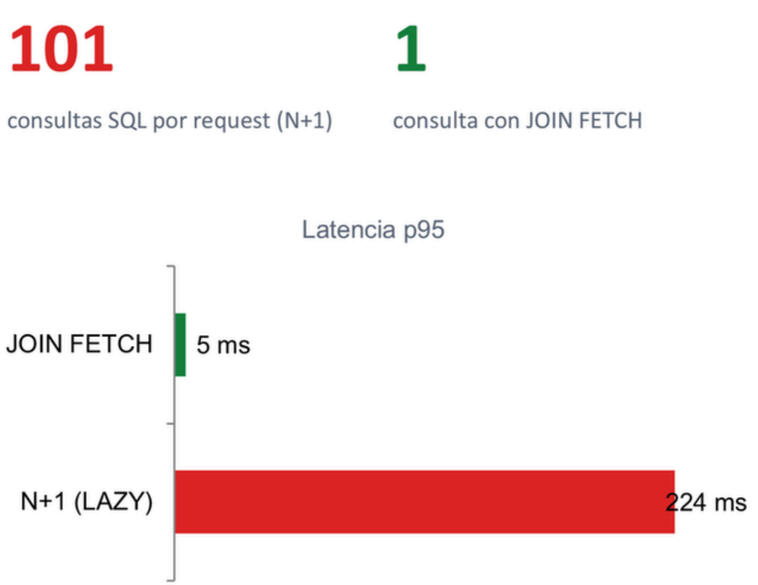
- La causa es la pasarela externa, no el servidor: máximo 5 de 50 hilos ocupados y CPU bajo 6%. k6 midió lo mismo desde el cliente (p99 de 1.559 ms).

CUELLO 2: EL PROBLEMA N+1

/api/pedidos/n-mas-1 hace 101 consultas SQL por request: 1 para los pedidos y 1 más por cada uno, porque la relación es LAZY.

Con JOIN FETCH es una sola consulta y el p95 baja de 224 ms a 5 ms: unas 45 veces más rápido.

Lo delató una métrica propia y el log JSON con mdc_sql_consultas = 101.



DE LA ALERTA AL LOG

La alerta TasaErroresAlta se disparó: falló el 4,4% de los pagos (213 de 4.801) y el umbral era 2%.

El dashboard acota el problema: solo /api/pagos tiene errores; pedidos y catálogo están en cero.

Loki explica el porqué: «La pasarela de pagos respondio timeout», y cada línea trae su request_id.





RED Y USE: QUÉ MEDIR

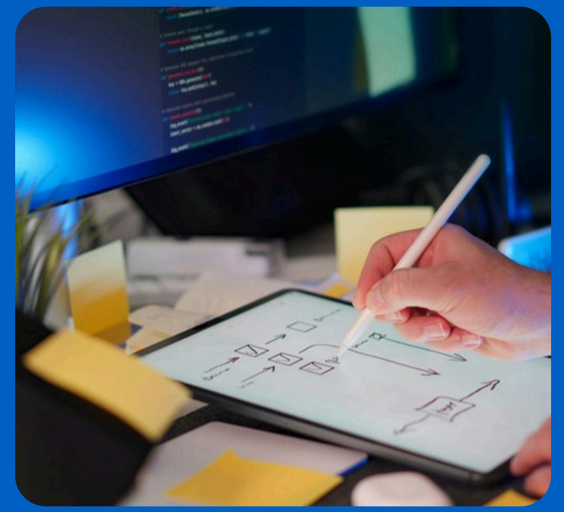
- RED, para servicios: Rate, Errors, Duration. Es la vista del usuario. En la demo marcó /pagos (cola y errores) y /pedidos/n-mas-1 (lento).

- USE, para recursos: Utilization, Saturation, Errors. La JVM estaba sana: heap bajo 145 MiB, 5 de 50 hilos y CPU bajo 6%. Escalar servidores no lo habría arreglado.

BUENAS PRÁCTICAS

- Logs en JSON con contexto (request_id, uri, duración) y percentiles con histogramas: SLOs en p95 y p99, nunca en el promedio.
- Alerta por síntomas que ve el usuario (errores, latencia) e investiga por causas. Etiquetas de baja cardinalidad: /pedidos/{id}, nunca el id.
- Mide también el negocio (pagos aprobados y fallidos) y versiona dashboards, datasources y alertas en Git.





ERRORES COMUNES

CARDINALIDAD

userId o requestId como etiqueta: millones de series y Prometheus se cae. Lo mismo pasa con Loki.

RUIDO

Alertas sin acción clara que el equipo termina silenciando, y dashboards de 40 paneles que nadie mira.

CEGUERA

Confiar en el promedio (103 ms con un p99 de 1.574 ms) e instrumentar solo después del incidente.

TALLER: AHORA LO ENCUENTRAN USTEDES

- Minutos 0 a 6: abren un GitHub Codespace (o Docker local), corren docker compose up y lanzan la carga con k6.
- Minutos 6 a 16: tres misiones. La cola de latencia (PromQL), cazar el N+1 (LogQL) y de la alerta al request_id.
- Minutos 16 a 20: reto. Crear una alerta de p99 mayor a 1 s y verla dispararse. La guía paso a paso está en TALLER.md.

¿PREGUNTAS?

Gracias